

AiSD Lista 1

Dominik Kowalczyk

12 listopada 2024

1 Algorytm sortowania INSERTION SORT (przez wstawianie)

W podstawowej wersji (bez modyfikacji) algorytmu sortowania przez wstawienie wyróżniamy następujący fragment kodu:

Listing 1: Kod dla funkcji sortującej przez wstawienie

```
1 void INSERTION_SORT(float A[], int n) {  
2     for (int i = 1; i < n; i++) {  
3         float key = A[i];  
4         int j = i - 1;  
5         while (j >= 0 && A[j] > key) {  
6             A[j + 1] = A[j];  
7             j--;  
8         }  
9         A[j + 1] = key;  
10    }  
11 }
```

Powyższy fragment kodu zaczyna się od funkcji `for`, która przechodzi kolejno po każdym elemencie tablicy `A`. Naszą iterację zaczynamy od drugiego elementu tablicy (z indeksem 1) (zakładając, że 0 jest już posortowany). Zmienna `key` umożliwia przechowanie wartości wstawianego elementu zapobiegając nadpisaniu oraz pozwalając na porównywanie tej wartości z kolejnymi elementami posortowanego ciągu. Gdy napotkany element w posortowanej części jest większy od wstawianego, przesuwamy go o jedną pozycję w prawo, aby zrobić miejsce na wstawienie nowego elementu. Jeśli element w posortowanej części jest mniejszy lub równy wstawianemu, wówczas zapisujemy wstawiany element na kolejnej pozycji.

Modyfikacja INSERTION SORTA polega na wstawianiu "na raz" dwóch kolejnych elementów tablicy. Zatem duża część kodu działa w dość analogiczny sposób.

Listing 2: Fragment 1 funkcji sortującej przez wstawienie z modyfikacją

```
1 void INSERTION_SORT_MOD(float A[], int n) {  
2     for (int s = 1; s < n - 1; s += 2) {  
3         float pierwszy = A[s];  
4         float drugi = A[s + 1];  
5         if (pierwszy > drugi) {  
6             swap(pierwszy, drugi);  
7         }  
8         int k = s - 1;  
9         while (k >= 0 && A[k] > pierwszy) {  
10             A[k + 1] = A[k];  
11             k--;  
12         }  
13         A[k + 1] = pierwszy;  
14         A[k + 2] = drugi;  
15     }  
16 }
```

Pętla zaczyna się od indeksu 1 i iteruje z krokiem +2, co pozwala jednocześnie rozpatrywać dwie wartości – *pierwszy* i *drugi*. Następnie porównujemy je za pomocą funkcji if i jeśli *pierwszy* jest większy zamieniamy miejscami. Funkcja while działa już na tej samej zasadzie co w kodzie bez modyfikacji, (opis wyżej).

Listing 3: Fragment 2 funkcji sortującej przez wstawienie z modyfikacją

```

1  if (n % 2 == 0) {
2      float ostatni = A[n - 1];
3      int k = n - 2;
4      while (k >= 0 && A[k] > ostatni) {
5          A[k + 1] = A[k];
6          k--;
7      }
8      A[k + 1] = ostatni;
9  }
```

Przedstawiona modyfikacja jednak wymaga rozważenia przypadku, czy tablica A ma długość parzystą, czy nieparzystą. Problem pojawia się gdy n jest parzyste, gdyż ostatni sortowany element nie ma pary ($ostatni = A[n - 1]$). Dla k przypisujemy przedostatni indeks tablicy, które też będzie miejscem rozpoczęcia wstawiania elementu $ostatni$. Pętla *while* działa identycznie jak ta w podstawowym INSERTION SORT, z uwagą, że naszym *key* jest *ostatni*.

2 Algorytm sortowania MERGE SORT (przez scalenie)

Listing 4: Fragment 1 funkcji sortującej przez scalenie bez modyfikacji

```

1  void MERGE(float A[], int p, int s, int k) {
2      int n1 = s - p + 1;
3      int n2 = k - s;
4      float L[n1 + 1];
5      float P[n2 + 1];
6      L[n1] = numeric_limits<float>::infinity();
7      P[n2] = numeric_limits<float>::infinity();
8  }
```

W przypadku MERGE SORTA ważną rolę odgrywa funkcja MERGE, na której na dobrą sprawę zbudowany jest MERGE SORT. MERGE przyjmuje cztery argumenty: p - początek, s - środek oraz k - koniec. Za ich pomocą dzielimy tablicę na dwie części: lewą ($n1 = s - p + 1$) oraz prawą ($n2 = k - s$). Definiujemy funkcje pomocnicze L i P o pojemności kolejno $n1 + 1$ oraz $n2 + 1$. Następnie

ustawiamy ostatni element na "napewno większy" od największego elementu tablicy $A[]$.

Listing 5: Fragment 2 funkcji sortującej przez scalenie bez modyfikacji

```
1 int i = 0;
2 int j = 0;
3 for (int l = p; l <= k; l++) {
4     if (L[i] <= P[j]) {
5         A[l] = L[i];
6         i++;
7     } else {
8         A[l] = P[j];
9         j++;
10    }
11 }
```

Po przypisaniu z lewej części do zmiennej L i z prawej części P resetujemy wskaźniki i oraz j (które wskazują naszą pozycję w kolejno w L i P).

Instrukcja $if(L[i] \leq P[j])$ sprawdza, czy bieżący element z L jest mniejszy lub równy bieżącemu elementowi z P .

Jeśli tak, to mniejszy element $L[i]$ jest wstawiany do tablicy $A[l]$, a wskaźnik i jest zwiększany o 1, by przejść do następnego elementu w L .

Jeśli nie, to mniejszy element $P[j]$ jest kopiowany do $A[l]$, a wskaźnik j jest zwiększany o 1, by przejść do następnego elementu w P .

Listing 6: Fragment 3 funkcji sortującej przez scalenie bez modyfikacji

```
1 void MERGESORT(float A[], int p, int k) {
2     if (p < k) {
3         int s = (p + k) / 2;
4         MERGESORT(A, p, s);
5         MERGESORT(A, s + 1, k);
6         MERGE(A, p, s, k);
7     }
8 }
```

Po przeanalizowaniu funkcji `MERGE`, możemy przejść do funkcji `MERGE SORT`, która wylicza srodek tablicy i wywołuje kolejno sortowanie z lewej strony, prawej strony po czym scala obie strony.

Listing 7: Fragment 1 funkcji sortującej przez scalenie z modyfikacją

```

1 void MERGEMODYFIKOWANY(float A[], int p, int s1, int s2, int k)
2     int n1 = s1 - p + 1;
3     int n2 = s2 - s1;
4     int n3 = k - s2;
5     float L[n1 + 1];
6     float S[n2 + 1];
7     float P[n3 + 1];
8     L[n1] = numeric_limits<float>::infinity();
9     S[n2] = numeric_limits<float>::infinity();
10    P[n3] = numeric_limits<float>::infinity();
11 }

```

Modyfikacja MERGE SORTA polegała na podzieleniu tablicy na trzy części zamiast dwóch, więc cały kod opiera się i ma bardzo podobną budowę do podstawowego MERGE SORTA. Zamiast dwóch części stworzymy trzy:

$$n1 = s1 - p + 1$$

$$n2 = s2 - s1$$

$$n3 = k - s2$$

a jako, że rozdzielamy tablicę. Pozostała reszta MERGA MODYFIKOWANEGO jak i MERGE SORTA MODYFIKOWANEGO przebiega bardzo podobnie. Ze względu na podział na trzy zamiast dwie części otrzymujemy dodatkową pętlę if przypisującą elementy ze środka do S:

Listing 8: Fragment 2 funkcji sortującej przez scalenie z modyfikacją

```

1 for (int j = 0; j < n2; j++){
2     S[j] = A[s1 + 1 + j];
3 }

```

oraz dodatkowy warunek na sortowanie:

Listing 9: Fragment 3 funkcji sortującej przez scalenie z modyfikacją

```

1 else if (S[j] <= L[i] && S[j] <= P[m]) {
2     A[x] = S[j];
3     j++;
4 }

```

odpowiedzialne za prawidłowe umiejscowienie w posortowanej tablicy elementu wziętego za nasz środkowy poprzez sprawdzanie czy

jest mniejszy od elementu lewego oraz prawego (na takiej samej zasadzie jak w MERGE SORT).

3 Algorytm sortowania HEAPSORT (przez kopcowanie)

Przy HEAPSORT podobnie jak przy MERGESORT ciekawe rzeczy dzieją się w funkcjach pomocniczych. Przeanalizujmy zatem HEAPIFY (na kopcach binarnych).

Listing 10: Fragment 2 funkcji sortującej przez kopcowanie bez modyfikacji

```
1 int LEWA(int i) {  
2     return 2 * i + 1;  
3 }  
4 int PRAWA(int i) {  
5     return 2 * i + 2;  
6 }
```

Kod powyżej definiuje pozycje lewego "dziecka" oraz prawego "dziecka" w kopcu binarnym.

Listing 11: Fragment 1 funkcji sortującej przez kopcowanie bez modyfikacji

```
1 void HEAPIFY(float A[], int i, int n) {  
2     int l = LEWA(i);  
3     int p = PRAWA(i);  
4     int naj = i;  
5  
6     if (l < n && A[l] > A[i]) {  
7         naj = l;  
8     }  
9  
10    if (p < n && A[p] > A[naj]) {  
11        naj = p;  
12    }  
13  
14    if (naj != i) {  
15        swap(A[i], A[naj]);  
16        HEAPIFY(A, naj, n);  
17    }  
18 }
```

Argumentami jest tablica $A[]$, która reprezentuje nasz "kopiec", i która oznacza indeks od którego zaczynamy "naprawę kopca" oraz n odpowiedzialna za długość tablic.

- $l = \text{LEWA}(i)$
Oblicza indeks lewego dziecka węzła i i przypisuje go do zmiennej l . Używa funkcji **LEWA**, która zwraca $2 \cdot i$.
- $p = \text{PRAWA}(i)$
Oblicza indeks prawego dziecka węzła i i przypisuje go do zmiennej p . Używa funkcji **PRAWA**, która zwraca $2 \cdot i + 1$.
- $naj = i$
Ustawia zmienną naj (przechowującą indeks największego elementu) na i . Zakładamy na początku, że węzeł i jest największy wśród siebie i swoich dzieci.
- if ($l < n$ and $A[l] > A[i]$) { $naj = l$; }
Sprawdza, czy lewe dziecko l znajduje się w zakresie kopca ($l < n$), a wartość w tablicy A pod indeksem l jest większa niż wartość w $A[i]$.
Jeśli oba warunki są spełnione, to zmienna naj zostaje ustawiona na l , co oznacza, że największy element jest teraz na pozycji lewego dziecka.

To samo robimy dla p , tylko zmieniamy $A[i]$ na $A[naj]$, gdyż w naturalny sposób przez pierwszego ifa wartość największa mogła się zmienić.

Na sam koniec sprawdzamy, czy znaleźliśmy element większy od naszego początkowego, jeśli tak to zamieniamy miejscami je miejscami. Jeszcze wywołujemy **HEAPIFY** rekurencyjnie na poddrzewie naj , aby upewnić się, że poddrzewo również spełnia własność kopca.

Listing 12: Fragment 2 funkcji sortującej przez kopcowanie bez modyfikacji

```

1 void BUILD_HEAP(float A[], int n) {
2     for (int i = n / 2 - 1; i >= 0; i--) {
3         HEAPIFY(A, i, n);
4     }
5 }
6
7 void HEAPSORT(float A[], int n) {
```

```

8      BUILD_HEAP(A, n);
9
10     for (int i = n - 1; i >= 1; i--) {
11         swap(A[0], A[i]);
12         n--;
13         HEAPIFY(A, 0, n);
14     }
15 }

```

W BUILD_HEAPIE inicjujemy pętlę for, która iteruje od środkowego elementu tablicy ($n/2 - 1$) wstecz do pierwszego elementu (indeks 0). Dzięki takiemu iterowaniu nasza funkcja HEAPIFY może poprawnie naprawić kopiec dla każdego poddrzewa, budując kopiec od dołu do góry.

W HEAPSORCIE:

- for ($i = n - 1; i \geq 1; i--$) {
Inicjuje pętlę, która iteruje od ostatniego elementu tablicy $A[n-1]$ wstecz do drugiego elementu $A[1]$.
przesuwając największy element kopca do jego końcowej pozycji.
- swap($A[0], A[i]$);
Zamienia miejscami elementy $A[0]$ (korzeń kopca, największy element) oraz $A[i]$ (bieżący koniec kopca), dzięki czemu największy element trafia na właściwe miejsce.

Listing 13: Fragment 1 funkcji sortującej przez kopcowanie z modyfikacją

```

1  int LEWAT(int i) {
2      return 3 * i + 1;
3  }
4  int SRODEK_T(int i) {
5      return 3 * i + 2;
6  }
7  int PRAWAT(int i) {
8      return 3 * i + 3;
9  }
10 void HEAPIFY_T(float A[], int i, int n) {
11     int l = LEWAT(i);

```



```

12     int s = SRODEK_T(i);
13     int p = PRAWA_T(i);
14     int naj = i;
15     if (l < n && A[l] > A[i]) {
16         naj = l;
17     }
18     if (s < n && A[s] > A[naj]) {
19         naj = s;
20     }
21     if (p < n && A[p] > A[naj]) {
22         naj = p;
23     }
24     if (naj != i) {
25         swap(A[i], A[naj]);
26         HEAPIFY_T(A, naj, n);
27     }
28 }

```

Modyfikacja algorytmu HEAPSORT miała polegać na użyciu kopców binarnych kopce ternarne. Ta modyfikacja najbardziej wpływa na HEAPIFY, gdyż oprócz lewego oraz prawego "dziecka" mamy również środkowe. Tak więc teraz:

- LEWA T zwraca $3 * i + 1$
- SRODEK T zwraca $3 * i + 2$
- PRAWA T zwraca $3 * i + 3$

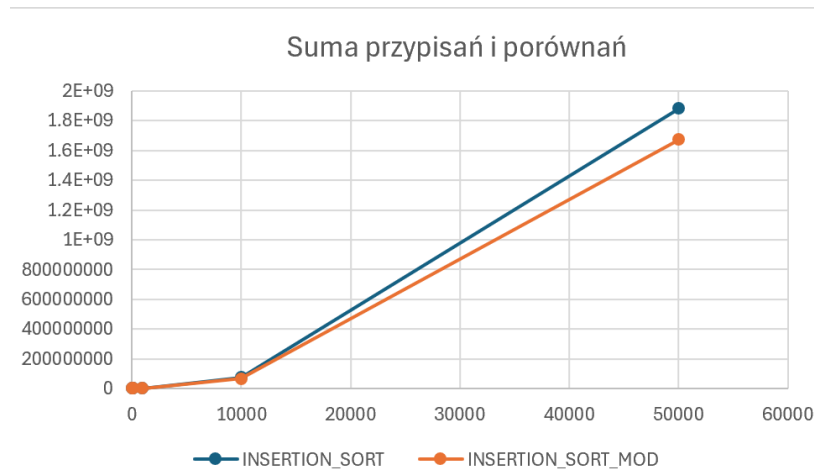
Więc w zmodyfikowanej wersji HEAPSORTA, w HEAPIFY występuje dodatkowa zmienna s , która przy porównywaniu wartości dodaje jedną dodatkową pętlę (sytuacja podobna jak w modyfikacji MERGE SORT).

4 Porównania

4.1 INSERTIONSORT

INSERTION_SORT					INSERTION_SORT_MOD			
ilość elementów	przypisania	porównania	suma		ilość elementów	przypisania	porównania	suma
10	63	27	90		10	63	24	87
100	4775	2338	7113		100	4775	1709	6484
1000	508927	253964	762891		1000	508927	169489	678416
10000	50020203	25005102	75025305		10000	50020203	16701567	66721770
50000	1254176605	627063303	1881239908		50000	1254176605	417993199	1672169804

Rysunek 1: Porównanie ilości porównań i przypisań w INSERTION SORT oraz INSERTION SORT MOD.



Rysunek 2: Wykres ilości porównań i przypisań w INSERTION SORT oraz INSERTION SORT MOD.

Czas wykonywania sortowania INSERTION SORT (średnia dla 10 sortowań):

dla 10000 elementów - 110,70ms

dla 50000 elementów - 2471,50ms

Czas wykonania sortowania INSERTION SORT MOD (średnia dla 10 losowań):

dla 10000 elementów - 106,40ms

dla 50000 elementów - 2413,50ms

Obydwa algorytmy mają bardzo zbliżoną złożoność, która minimalnie jest większa dla zmodyfikowanej wersji insertion sort. Pod

względem ilości czasu na sortowanie również wypadają bardzo podobnie, ze wskazaniem na modyfikację. Warto zauważyć, że różnica pomiędzy dwoma wersjami kodu występuje tylko w przypadku ilości powtórzeń, ilość przypisań jest stała. Zatem możemy stwierdzić, że nasza modyfikacja polegająca na sprawdzaniu dwóch kolejnych elementów "na raz" daje nam zmniejszenie ilości sprawdzanych warunków w pętlach.

Złożoność:

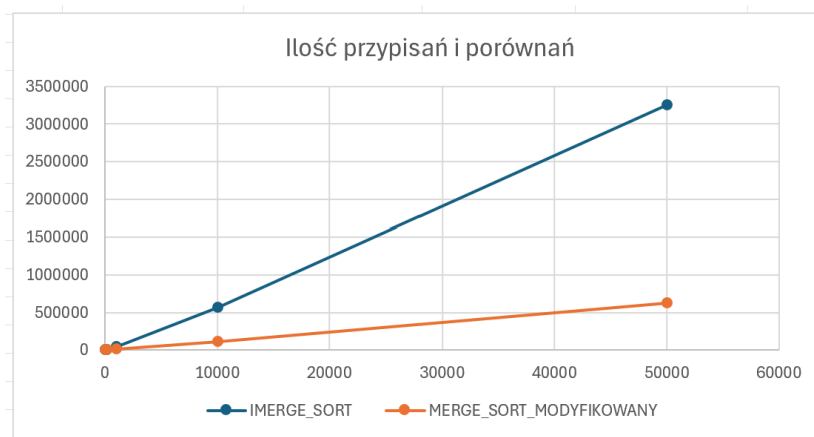
- Złożoność pesymistyczna: $O(n^2)$,

4.2 MERGESORT

MERGE_SORT			
ilość elementów	przypisania	porównania	suma
10	168	43	211
100	2591	771	3362
1000	33987	10976	44962
10000	428231	143615	569846
50000	2420871	834463	3255334
	2883848	989867	

MERGE_SORT_MODYFIKOWANY			
ilość elementów	przypisania	porównania	suma
10	23	27	50
100	203	497	700
1000	2003	7177	9180
10000	20003	93597	113600
50000	100003	529524	629527

Rysunek 3: Porównanie ilości porównań i przypisań w MERGE SORT oraz MERGE SORT MODYFIKOWANY.



Rysunek 4: Wykres ilości porównań i przypisań w MERGE SORT oraz MERGE SORT MODYFIKOWANY.

Czas wykonywania sortowania MERGE SORT (średnia dla 10 losowań):
dla 10000 elementów - 1,80ms

dla 50000 elementów - 10,70ms

Czas wykonania sortowania MERGE SORT MODYFIKOWANY (średnia dla 10 losowań):

dla 10000 elementów - 1,90ms

dla 50000 elementów - 9,00ms

W tym przypadku zdecydowanie bardziej wydajniejszym i lżejszym algorytmem jest merge sort z modyfikacją. Zarówno ilość przypisań jak i porównań jest znacznie mniejsza, co stwierdza jednoznacznie poprawność oraz optymalność modyfikacji. Pod względem czasu czasy ponownie są dosyć zbliżone do siebie, jednakże dla bardziej rozległych tablic wygrywać zdaje się zmodyfikowany merge sort. Oba zaimplementowane algorytmy merge sorta są zdecydowanie szybsze od ukazanych insertion sortów. Insertion sorty mają mniejszą ilość przypisań niż merge sort dla bardzo małych tablic, jednakowoż dla większych merge sort staje się o wiele wydajniejszym i lżejszym algorytmem.

- **Złożoność pesymistyczna:** $O(n \log n)$,

4.3 HEAPSORT

HEAPSORT			
ilość elementów	przypisania	porównania	suma
10	230	57	287
100	4416	1352	5768
1000	65527	21753	87280
10000	872586	300730	1173326
50000	5120725	1784719	6915444
	6063494	2118611	

HEAP_SORT_T			
ilość elementów	przypisania	porównania	suma
10	193	38	231
100	3616	961	4577
1000	51631	15001	66632
10000	680540	208905	889505
50000	3870183	1250621	5221004

Rysunek 5: Porównanie ilości porównań i przypisań w HEAPSORT oraz HEAP SORT T.

Czas wykonywania sortowania HEAPSORT (średnia dla 10 losowań):

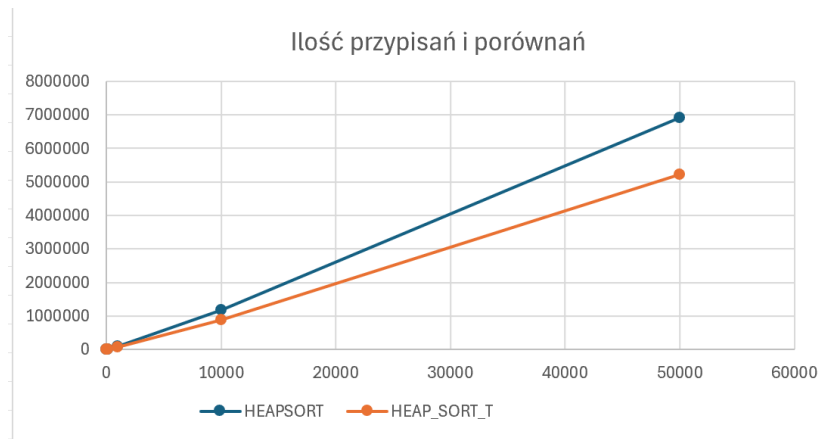
dla 10000 elementów - 4,00ms

dla 50000 elementów - 22,40ms

Czas wykonania sortowania HEAP SORT T (średnia dla 10 losowań):

dla 10000 elementów - 3,70ms

dla 50000 elementów - 19,30ms



Rysunek 6: Wykres ilości porównań i przypisań w HEAPSORT oraz HEAP SORT T.

W przypadku algorytmu heapsort widać różnicę złożoności algorytmu po modyfikacji, a przed. Heapsort z kopcami ternarnymi jest wydajniejszy oraz szybszy dla większych zbiorów elementów. Dla większych tablic ilość porównań jak i przypisań jest mniejsza dla kopca ternarnego, co ostatecznie pozwala nam stwierdzić, że modyfikacja daje nam większą efektywność algorytmu.

- **Złożoność pesymistyczna:** $O(n \log n)$,

5 Wnioski

Podsumowując wszystkie zaimplementowane modyfikacje do algorytmów sortujących przez wstawianie, scalanie oraz kopcowanie okazały się wydajniejsze, szybsze, lżejsze obliczeniowo, zwiększając tym samym całkowitą efektywność pierwotnych algorytmów. Porównując jednakże każdy algorytm z każdym, to zwycięskim, najlepszym algorytmem nazwiemy zmodyfikowany merge sort, który "wygrywa" z pozostałymi pod każdym względem. Natomiast najmniej optymalnym, najdłużej działającym i najcięższym obliczeniowo algorytmem będzie insertion sort (bez modyfikacji), który odstaje zdecydowanie od zaimplementowanych algorytmów sortowania przez scsłanie oraz kopcowanie. HEAPSORT jest dosyć zbliżony do merge sorta, co pokrywa się z teorią dotyczącą złożoności pesymistycznych tych algorytmów.